In [2]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

In [3]:
```
# ── Load data ──────────────────────────────────────────────────────────
df = pd.read_csv('household_10k.csv')
```

In [4]:
```
# ── Step 1: Remove duplicates ──────────────────────────────────────────
df = df.drop_duplicates()
print(f"[2] After removing duplicates: {df.shape}")
```

```
[2] After removing duplicates: (9305, 4)
```

In [5]:
```
# ── Step 2: Invalid / out-of-range values → NaN ────────────────────────
valid_ranges = {
    'Household_Size':    (1, 10),
    'Seasonal_Index':    (1, 4),
    'Garden_Area':       (0, 2000),
    'Daily_Liters_Used': (50, 5000),
}

for col, (lo, hi) in valid_ranges.items():
    mask = df[col].notna() & ((df[col] < lo) | (df[col] > hi))
    print(f"   Invalid values in '{col}': {mask.sum()}")
    df[col] = df[col].where(~mask, other=np.nan)

print(f"[3] Missing values after range check:\n{df.isnull().sum()}\n")
```

```
  Invalid values in 'Household_Size': 113
  Invalid values in 'Seasonal_Index': 113
  Invalid values in 'Garden_Area': 122
  Invalid values in 'Daily_Liters_Used': 124
[3] Missing values after range check:
Household_Size       608
Seasonal_Index       606
Garden_Area          584
Daily_Liters_Used    513
dtype: int64
```

In [6]:
```
# ── Step 3: Impute missing values ──────────────────────────────────────
# Categorical-like columns → mode
for col in ['Household_Size', 'Seasonal_Index']:
    df[col] = df[col].fillna(df[col].mode()[0])
# Continuous columns → median
for col in ['Garden_Area', 'Daily_Liters_Used']:
    df[col] = df[col].fillna(df[col].median())

print(f"[4] Missing values after imputation:\n{df.isnull().sum()}\n")
```

```
[4] Missing values after imputation:
Household_Size       0
Seasonal_Index       0
Garden_Area          0
Daily_Liters_Used    0
dtype: int64
```

In [7]:
```
# ── Step 4: Convert to proper integer types ──────────────────────────
df['Household_Size']    = df['Household_Size'].round().astype('Int64')
df['Seasonal_Index']    = df['Seasonal_Index'].round().astype('Int64')
df['Garden_Area']       = df['Garden_Area'].round().astype('Int64')
df['Daily_Liters_Used'] = df['Daily_Liters_Used'].round().astype('Int64')

print(f"[5] Data types after conversion:\n{df.dtypes}\n")
```

```
[5] Data types after conversion:
Household_Size       Int64
Seasonal_Index       Int64
Garden_Area          Int64
Daily_Liters_Used    Int64
dtype: object
```

In [8]:
```
# ── Step 5: IQR-based outlier removal ────────────────────────────────
before = len(df)
for col in ['Garden_Area', 'Daily_Liters_Used']:
    Q1  = df[col].quantile(0.25)
    Q3  = df[col].quantile(0.75)
    IQR = Q3 - Q1
    df  = df[(df[col] >= Q1 - 1.5 * IQR) & (df[col] <= Q3 + 1.5 * IQR)]

print(f"[6] Rows removed by IQR filter: {before - len(df)}")
print(f"    Shape after IQR removal: {df.shape}")

df = df.reset_index(drop=True)
```

```
[6] Rows removed by IQR filter: 0
    Shape after IQR removal: (9305, 4)
```

In [9]:
```
# ── Step 6: Min-Max scaling ──────────────────────────────────────────
scaler = MinMaxScaler()
df[['Garden_Area_scaled', 'Daily_Liters_scaled']] = scaler.fit_transform(
    df[['Garden_Area', 'Daily_Liters_Used']]
).round(4)

print(f"[7] Min-Max scaled columns added: Garden_Area_scaled, Daily_Liters_scaled")
```

```
[7] Min-Max scaled columns added: Garden_Area_scaled, Daily_Liters_scaled
```

In [10]:
```
# ── Step 7: One-hot encode Seasonal_Index ────────────────────────────
df = pd.get_dummies(df, columns=['Seasonal_Index'], prefix='Season', dtype=int)

print(f"[8] After one-hot encoding: {df.shape}")
print(f"    Columns: {list(df.columns)}")
```

```
[8] After one-hot encoding: (9305, 9)
    Columns: ['Household_Size', 'Garden_Area', 'Daily_Liters_Used', 'Garden_Area_sca
led', 'Daily_Liters_scaled', 'Season_1', 'Season_2', 'Season_3', 'Season_4']
```

In [11]:
```python
# ── Final summary ──────────────────────────────────────────────────────────
print(f"\n{'='*50}")
print(f"FINAL DATASET")
print(f"{'='*50}")
print(f"Shape       : {df.shape}")
print(f"Missing vals: {df.isnull().sum().sum()}")
print(f"\nSample:\n{df.head()}")

# ── Save ───────────────────────────────────────────────────────────────────
df.to_csv('household_preprocessed.csv', index=False)
print(f"\nSaved to: household_preprocessed.csv")
```

```
==================================================
FINAL DATASET
==================================================
Shape       : (9305, 9)
Missing vals: 0

Sample:
   Household_Size  Garden_Area  Daily_Liters_Used  Garden_Area_scaled  \
0               1           20                110              0.0000
1               2           40                170              0.0870
2               3           70                260              0.2174
3               4          100                340              0.3478
4               5          150                520              0.5652

   Daily_Liters_scaled  Season_1  Season_2  Season_3  Season_4
0               0.0414         1         0         0         0
1               0.1271         1         0         0         0
2               0.2557         0         1         0         0
3               0.3700         0         1         0         0
4               0.6271         0         0         1         0

Saved to: household_preprocessed.csv
```

In [15]:
```python
# Applying Standard Scaler
from sklearn.preprocessing import StandardScaler
X = df[['Household_Size', 'Garden_Area']]
y = df['Daily_Liters_Used']
scaler = StandardScaler()
X_scaled_array = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled_array, columns=X.columns)
final_df = X_scaled.copy()
final_df['Daily_Liters_Used'] = y.reset_index(drop=True)
print("Scaled Features (First 5 rows):")
print(final_df.head())
```

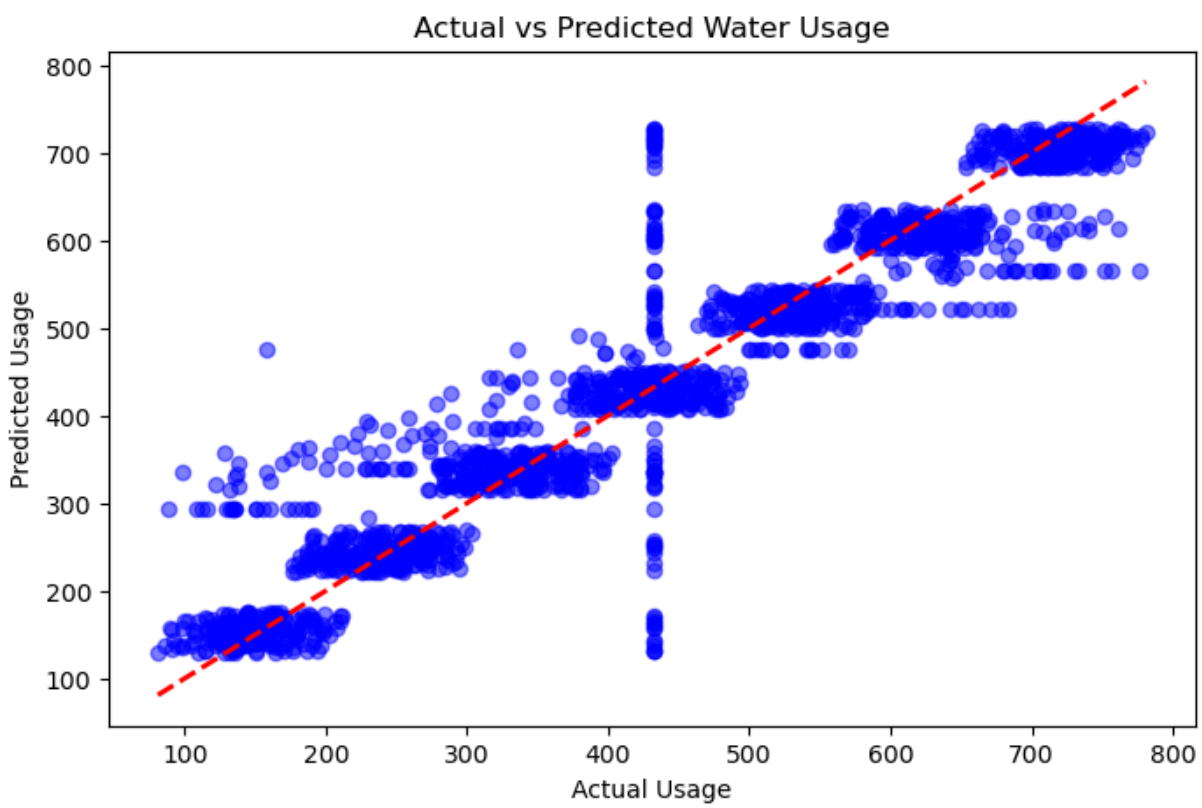
```
Scaled Features (First 5 rows):
   Household_Size  Garden_Area  Daily_Liters_Used
0       -1.575279    -1.781101                110
1       -1.061811    -1.440284                170
2       -0.548343    -0.929058                260
3       -0.034875    -0.417832                340
4        0.478593     0.434212                520
```

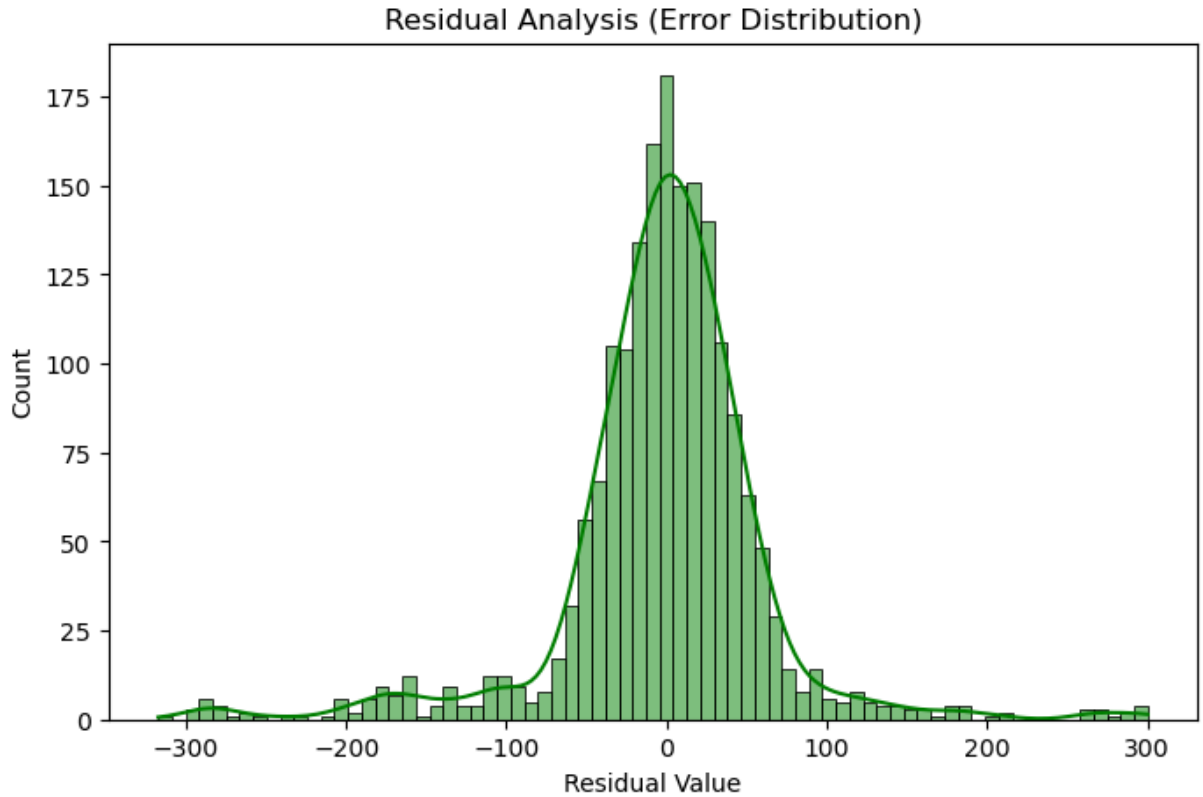
In [18]:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load the dataset
df = pd.read_csv('household_preprocessed.csv')

# --- STEP 1: PREPROCESSING (Cleaning & Outlier Removal) ---
df_cleaned = df.dropna().copy()
cols = ['Household_Size', 'Garden_Area', 'Daily_Liters_Used']
for col in cols:
    df_cleaned = df_cleaned[df_cleaned[col] > 0]

def remove_outliers(data, columns):
    for col in columns:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        data = data[(data[col] >= Q1 - 1.5*IQR) & (data[col] <= Q3 + 1.5*IQR)]
    return data

df_cleaned = remove_outliers(df_cleaned, cols)

# --- STEP 2: SPLITTING & SCALING ---
X = df_cleaned[['Household_Size', 'Garden_Area']]
y = df_cleaned['Daily_Liters_Used']

# Split into 80% Training and 20% Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Apply StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- STEP 3: MODEL BUILDING ---
model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Make Predictions
y_pred = model.predict(X_test_scaled)

# --- STEP 4: EVALUATION ---
```

```
print(f"R-squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")

# --- STEP 5: RESIDUAL ANALYSIS (Visualization) ---
residuals = y_test - y_pred

# Plotting Actual vs Predicted
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, color='blue', alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Actual vs Predicted Water Usage')
plt.xlabel('Actual Usage')
plt.ylabel('Predicted Usage')
plt.show()

# Plotting Residual Distribution
plt.figure(figsize=(8, 5))
sns.histplot(residuals, kde=True, color='green')
plt.title('Residual Analysis (Error Distribution)')
plt.xlabel('Residual Value')
plt.show()
```

```
R-squared Score: 0.8849
Mean Squared Error: 4094.19
```

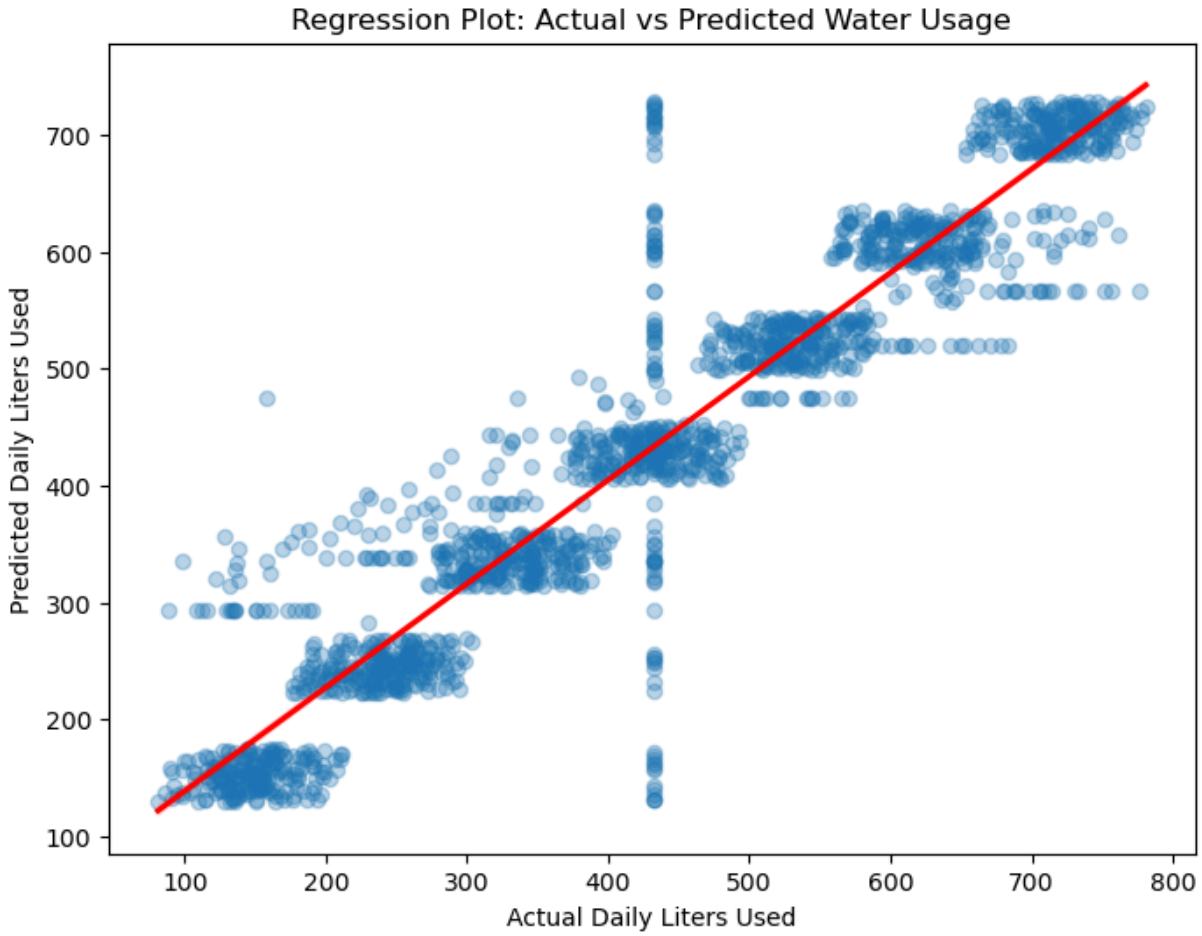


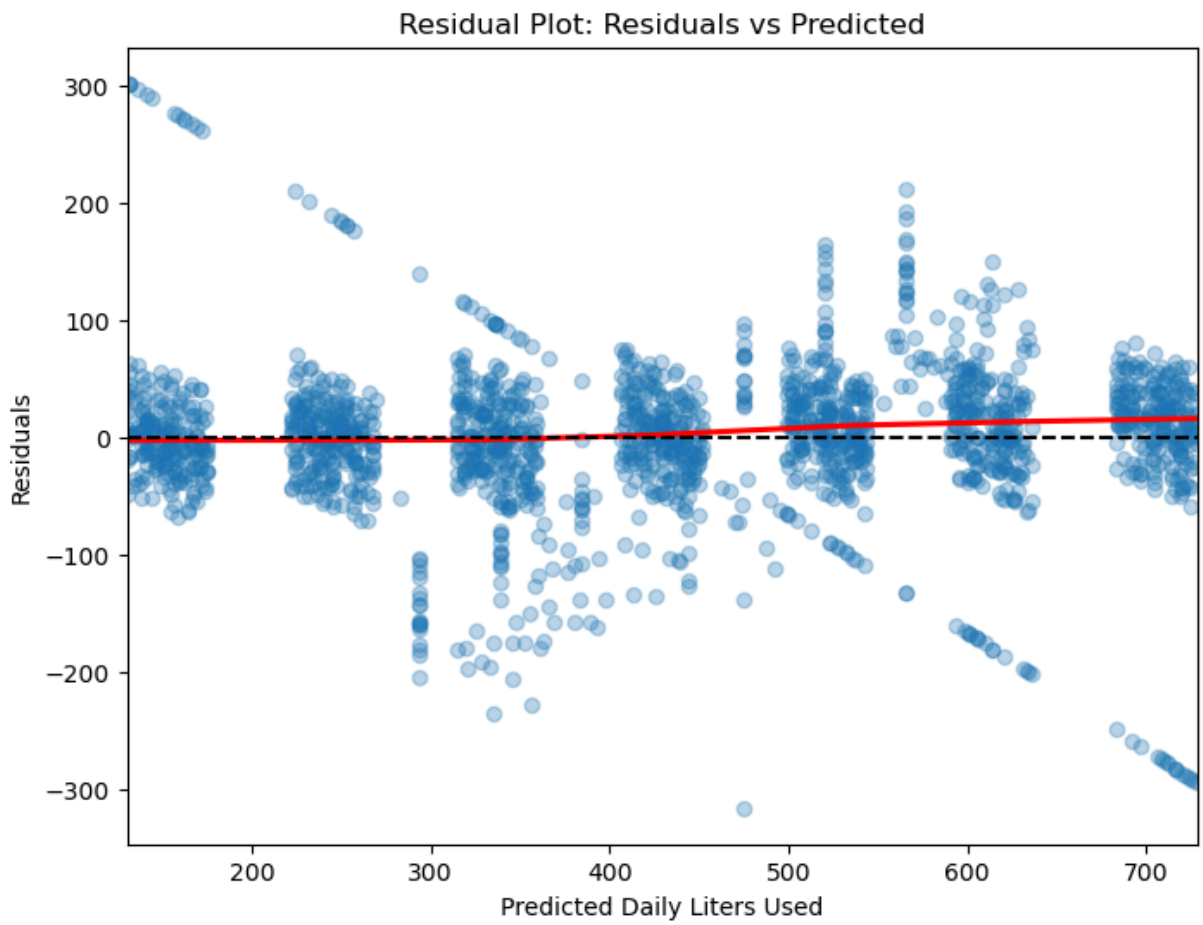


In [20]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 1. Load the dataset
df = pd.read_csv('household_preprocessed.csv')

# --- STEP 1: PREPROCESSING ---
df_cleaned = df.dropna().copy()
cols = ['Household_Size', 'Garden_Area', 'Daily_Liters_Used']
for col in cols:
    df_cleaned = df_cleaned[df_cleaned[col] > 0]

# Outlier removal
def remove_outliers(data, columns):
    for col in columns:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        data = data[(data[col] >= Q1 - 1.5*IQR) & (data[col] <= Q3 + 1.5*IQR)]
    return data

df_cleaned = remove_outliers(df_cleaned, cols)

# --- STEP 2: SPLITTING & SCALING ---
X = df_cleaned[['Household_Size', 'Garden_Area']]
```

```
y = df_cleaned['Daily_Liters_Used']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- STEP 3: MODEL BUILDING ---
model = LinearRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

# --- STEP 4: EVALUATION METRICS ---
print(f"Mean Absolute Error (MAE): {mean_absolute_error(y_test, y_pred):.2f}")
print(f"Mean Squared Error (MSE): {mean_squared_error(y_test, y_pred):.2f}")
print(f"R-squared Score: {r2_score(y_test, y_pred):.4f}")

# --- STEP 5: VISUALIZATIONS ---

# A. Regression Plot (Actual vs Predicted)
plt.figure(figsize=(8, 6))
sns.regplot(x=y_test, y=y_pred, scatter_kws={'alpha':0.3}, line_kws={'color':'red'}
plt.title('Regression Plot: Actual vs Predicted Water Usage')
plt.xlabel('Actual Daily Liters Used')
plt.ylabel('Predicted Daily Liters Used')
plt.show()

# B. Residual Plot (Residuals vs Predicted)
plt.figure(figsize=(8, 6))
residuals = y_test - y_pred
sns.residplot(x=y_pred, y=residuals, lowess=True, scatter_kws={'alpha': 0.3}, line_
plt.title('Residual Plot: Residuals vs Predicted')
plt.xlabel('Predicted Daily Liters Used')
plt.ylabel('Residuals')
plt.axhline(y=0, color='black', linestyle='--')
plt.show()
```

```
Mean Absolute Error (MAE): 40.65
Mean Squared Error (MSE): 4094.19
R-squared Score: 0.8849
```


Regression Plot: Actual vs Predicted Water Usage
Predicted Daily Liters Used
Actual Daily Liters Used
100
200
300
400
500
600
700
800


Residual Plot: Residuals vs Predicted
Residuals
Predicted Daily Liters Used
300
200
100
0
−100
−200
−300
200
300
400
500
600
700

In [ ]:

In [ ]:

In [ ]: